

Experiment: Creating an HTTP Web Server Using Node.js

Aim

To build, configure, and execute a standalone HTTP web server using the built-in Node.js `http` module to handle client requests and deliver appropriate HTTP responses.

Software & Hardware Requirements

Requirement	Specification
Operating System	Windows 10/11, macOS, or Linux
Runtime Environment	Node.js (v16.x or higher)
Code Editor	Visual Studio Code, Sublime Text, or any text editor
Testing Client	Web Browser (Chrome, Firefox) or <code>curl</code>

Theory

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model that makes it suitable for lightweight, high-concurrency network applications.

Unlike traditional server environments (e.g., Apache or Nginx executing PHP/Python scripts), a Node.js application functions as its own web server using core system libraries:

- **`http` Module:** Node's built-in module that wraps low-level network streaming and HTTP parsing.
- `http.createServer([options][, requestListener])`: Instantiates an HTTP server instance. The listener callback executes on every incoming request, passing two core arguments:
 - `req (http.IncomingMessage)`: An event emitter carrying request headers, URL path, HTTP method, and incoming data streams.

- `res (http.ServerResponse)`: A writable stream used to set response status codes, HTTP headers, and stream content back to the client.
- `server.listen(port, [host], [callback])`: Binds the server process to a designated network interface and port to begin accepting connections.

Program Code (server.js)

```
// Import the built-in HTTP module
const http = require('http');

// Define network parameters
const HOSTNAME = '127.0.0.1';
const PORT = 3000;

// Create HTTP server instance
const server = http.createServer((req, res) => {
  // Log incoming client requests
  console.log(`Received ${req.method} request for: ${req.url}`);

  // Route handling
  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Welcome to the Node.js Web Server!\n');
  } else if (req.url === '/api/data' && req.method === 'GET') {
    const payload = JSON.stringify({
      status: 'success',
      timestamp: new Date().toISOString(),
      message: 'Server is running smoothly'
    });
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(payload);
  } else {
    // Fallback 404 Not Found
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 Not Found: The requested route does not exist.\n');
  }
});

// Bind server to port and host
server.listen(PORT, HOSTNAME, () => {
  console.log(`Server successfully started.`);
  console.log(`Running at http://${HOSTNAME}:${PORT}/`);
});
```

Procedure

1. Verify Node.js Installation: Prerequisite check.

Open your terminal or command prompt and verify the installation:

```
Bash
node -v
npm -v
```

Ensure a version string (e.g., v20.x.x) appears for both.

2.Initialize Project Directory:

Create an isolated directory for the lab and navigate into it:

```
Bash
mkdir node-server-lab
cd node-server
```

3.Create and Populate the Server File:

Create a file named `server.js` and paste the provided code using your text editor.

4.Start the Server Process:

Run the application directly via the Node runtime:

```
Bash
node server.js
```

Keep this terminal window open; it acts as the running server daemon.

5.Send Requests to Verify Endpoints:

Open a web browser or a second terminal window and test the endpoints:

- **Root Route:** Visit `[http://127.0.0.1:3000/]` (`http://127.0.0.1:3000/`)
- **JSON API Route:** Visit `[http://127.0.0.1:3000/api/data]` (`http://127.0.0.1:3000/api/data`)
- **Invalid Route:** Visit `[http://127.0.0.1:3000/unknown]` (`http://127.0.0.1:3000/unknown`)

Expected Output

Terminal Output (Server Console):

Plaintext

```
Server successfully started.  
Running at http://127.0.0.1:3000/  
Received GET request for: /  
Received GET request for: /api/data  
Received GET request for: /unknown
```

Client Responses (Browser or curl):

- GET [http://127.0.0.1:3000/] (http://127.0.0.1:3000/)

Plaintext

```
Welcome to the Node.js
```

- GET [http://127.0.0.1:3000/api/data] (http://127.0.0.1:3000/api/data)

JSON

```
{  
  "status": "success",  
  "timestamp": "2026-09-08T00:00:00.000Z",  
  "message": "Server is running smoothly"  
}
```

- GET [http://127.0.0.1:3000/unknown] (http://127.0.0.1:3000/unknown)

Plaintext

```
404 Not Found: The
```

The Node.js HTTP web server was successfully built and tested. The server proved capable of listening on port 3000, parsing client URLs, delivering plain text and JSON responses with appropriate headers, and returning standard 404 status codes for unmapped endpoints.

In native Node.js, the incoming `req` object is a readable stream. Because request bodies arrive in asynchronous binary chunks over the network, you must assemble these chunks into a complete buffer before parsing the data with `JSON.parse()`.

```
const http = require('http');

const PORT = 3000;
const MAX_PAYLOAD_SIZE = 1e6; // 1 MB payload limit

const server = http.createServer((req, res) => {
  // Route matching for POST requests
  if (req.method === 'POST' && req.url === '/api/users') {
    // Enforce Content-Type validation
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      res.writeHead(415, { 'Content-Type': 'application/json' });
      return res.end(JSON.stringify({ error: 'Unsupported Media Type: Expected application/json' }));
    }

    const chunks = [];
    let bytesReceived = 0;

    // Collect incoming byte chunks
    req.on('data', (chunk) => {
      bytesReceived += chunk.length;

      // Guard against large payloads (Denial of Service mitigation)
      if (bytesReceived > MAX_PAYLOAD_SIZE) {
        res.writeHead(413, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ error: 'Payload Too Large' }));
        req.destroy(); // Terminate the underlying socket
      } else {
        chunks.push(chunk);
      }
    });

    // Parse accumulated data once stream finishes
    req.on('end', () => {
      if (req.destroyed) return;

      const rawBody = Buffer.concat(chunks).toString('utf8');

      try {
        // Parse payload safely within a try/catch block
        const payload = rawBody ? JSON.parse(rawBody) : {};
      }
    });
  }
});
```

```

    // Process data and return response
    res.writeHead(201, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({
      status: 'success',
      received: payload
    }));
  } catch (err) {
    // Catch malformed JSON syntax errors
    res.writeHead(400, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({
      error: 'Bad Request: Malformed JSON syntax'
    }));
  }
});

req.on('error', (err) => {
  console.error('Request stream error:', err);
  res.writeHead(500, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Internal Server Error' }));
});

} else {
  res.writeHead(404, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Route Not Found' }));
}
});

server.listen(PORT, () => {
  console.log(`Server listening on http://localhost:${PORT}`);
});

```

Key Mechanics

- **Stream Accumulation:** `req.on('data')` receives instances of Node.js `Buffer`. Pushing them to an array and using `Buffer.concat(chunks).toString('utf8')` on the `end` event avoids truncating multi-byte characters (like emojis or non-ASCII text).
- **DoS Protection:** The `bytesReceived` counter tracks total memory usage in real time. Calling `req.destroy()` kills the incoming TCP socket immediately if an attacker attempts to flood the server.
- **Malformed JSON Handling:** `JSON.parse()` throws a synchronous runtime exception if provided with invalid JSON. Wrapping it inside a `try...catch` block prevents the entire Node.js server process from crashing.

Testing the Endpoint

Valid JSON Request:

Bash

```
curl -X POST http://localhost:3000/api/users \  
      -H "Content-Type: application/json" \  
      -d '{"name": "Jane Doe", "role": "Engineer"}'
```

JSON

```
{"status": "success", "received": {"name": "Jane Doe", "role": "Engineer"}}
```

Malformed JSON Request (Returns 400):

Bash

```
curl -X POST http://localhost:3000/api/users \  
      -H "Content-Type: application/json" \  
      -d '{"name": "Jane Doe",}'
```

JSON

```
{"error": "Bad Request: Malformed JSON syntax"}
```